Universidad de Buenos Aires
Facultad de Ciencias Exactas y Naturales
Departamento de Computación

Improving Community Membership Hiding via Deep Reinforcement Learning

Tesis de Licenciatura en Ciencias de la Computación

Dante Culaciati

Director: Esteban Feuerstein
Codirector: [Nombre del Codirector]
Buenos Aires, 2026

MEJORANDO EL OCULTAMIENTO DE MEMBRESÍA DE COMUNIDADES MEDIANTE APRENDIZAJE PROFUNDO POR REFUERZO

El problema de *Community Membership Hiding* (CMH) consiste en, dado un grafo, un algoritmo de detección de comunidades y un nodo objetivo u, sugerir un conjunto mínimo de reconexiones de aristas para que u deje de ser detectado como miembro de su comunidad original. Este problema es relevante en el contexto de privacidad en redes, donde los algoritmos de detección pueden revelar información sensible sobre los usuarios a partir de su posición estructural en el grafo.

Trabajos recientes han formulado CMH como un Proceso de Decisión de Markov resuelto mediante Aprendizaje Profundo por Refuerzo (DRL), utilizando una arquitectura Actor-Crítico. En este trabajo identificamos una limitación arquitectónica crítica del método original: el agente evalúa el grafo de forma global sin condicionar explícitamente en el nodo objetivo, lo que le impide aprender una política específica de ocultamiento.

Proponemos una arquitectura DRL novedosa que incorpora explícitamente la identidad del nodo objetivo como entrada de la red, utiliza *Graph Attention Networks v2* (GATv2) como codificador de grafos, y combina señales estáticas y dinámicas para representar fielmente el estado del grafo durante el proceso de reconexción. Extendemos este sistema de un solo agente a una *Arquitectura Dual*, que especializa agentes separados para la adición y eliminación de aristas, e introducimos una heurística de *Subgrafado Guiado por Política* que hace escalable el método en grafos de gran tamaño.

La evaluación experimental sobre los datasets estándar del área muestra que nuestra arquitectura supera consistentemente al método original y a los métodos de referencia basados en gradientes, logrando altas tasas de éxito de ocultamiento con una perturbación estructural mínima.

Palabras clave: detección de comunidades, ocultamiento de comunidades, aprendizaje por refuerzo profundo, redes de atención de grafos, privacidad en redes, GATv2, Actor-Crítico.

IMPROVING COMMUNITY MEMBERSHIP HIDING VIA DEEP REINFORCEMENT LEARNING

The *Community Membership Hiding* (CMH) problem asks: given a graph, a community detection algorithm, and a target node u, find a minimal set of edge rewirings such that u is no longer detected as a member of its original community. This problem arises naturally in the context of network privacy, where community detection algorithms can inadvertently expose sensitive user affiliations—such as political or ideological leanings—solely from the structural position of a node in the underlying graph.

Recent work has formulated CMH as a Markov Decision Process (MDP) solved via Deep Reinforcement Learning (DRL), employing an Actor-Critic architecture. While that approach demonstrated the viability of the paradigm, we identify a critical architectural limitation: the agent evaluates the network globally without explicitly conditioning on the identity of the target node, thereby learning a generic community-shuffling policy rather than a targeted hiding strategy.

In this thesis, we propose a novel DRL architecture that addresses these fundamental shortcomings. Our model explicitly incorporates the target node as a feature of the network representation, employs Graph Attention Networks v2 (GATv2) as the graph encoder to leverage dynamic, query-conditioned attention, and combines static and real-time structural signals to accurately track topological changes during rewiring. We further extend the single-agent system into a *Dual-Agent Architecture* that dedicates separate agents to edge addition and edge deletion, and introduce a *Policy-Guided Subgraphing* heuristic that makes the framework tractable on large-scale graphs.

Extensive experimental evaluation on the standard benchmark datasets of the field demonstrates that our architecture consistently outperforms both the original DRL agent and strong gradient-based baselines, achieving high hiding success rates with minimal structural perturbation.

Keywords: community detection, community membership hiding, deep reinforcement learning, graph attention networks, network privacy, GATv2, Actor-Critic.

ACKNOWLEDGEMENTS

[To be completed.]

CONTENTS

1. INTRODUCTION

The modern science of networks has brought significant advances to our understanding of complex systems. One of the most relevant structural features of graphs representing real-world phenomena, ranging from social media platforms to biological interaction networks, is their *community structure*: the tendency for nodes to cluster into densely connected groups that are only sparsely connected to the rest of the network. Community detection algorithms that uncover this structure have proven highly effective [9], and are now routinely deployed in large-scale platforms to characterize users, personalize content, and target advertisements.

However, this functionality carries a fundamental privacy risk. When a graph encodes social relationships, the community a node belongs to may inadvertently reveal sensitive affiliations (political leanings, religious beliefs, health conditions, or ideological associations) purely from its structural position, without any explicit disclosure of such information. This concern is not merely theoretical: community detection has been used in practice to identify political groups in online networks [18] and to profile individuals in social graphs [8].

To counter this, the concept of *Community Membership Hiding* (CMH) was introduced by Bernini, Silvestri, and Tolomei [2]. The goal of CMH is to strategically modify the topology of a network, by adding or removing a small number of edges adjacent to a target node u, so that u is no longer assigned to its original community by a detection algorithm, while keeping overall graph disruption minimal. The detection algorithm is treated as a *black box*, meaning that no knowledge of its internals is exploited.

Bernini et al. formulated CMH as a Markov Decision Process (MDP) and solved it via Deep Reinforcement Learning (DRL) using an Actor-Critic architecture (A2C). While that work demonstrated the viability of the DRL approach, it exhibits notable performance degradation on larger graphs, and, as we show in this thesis, harbors a critical architectural flaw: the agent evaluates the network *globally*, without explicitly conditioning on the identity of the target node. As a consequence, the agent cannot learn a policy that depends on who the target is; it instead learns a generic community-shuffling strategy that happens to displace the target node as a side effect.

[Figure placeholder: A graph with three communities (color-coded). The target node u (marked with a star) is currently in the blue community. After edge rewiring, u migrates to the red community.]

Fig. 1.1: A graph with three communities (color-coded). The target node u (marked with a star) is currently in the blue community. After edge rewiring, u migrates to the red community.

Contributions

In this thesis, we make the following contributions:

1. **Identification of an architectural limitation.** We formally demonstrate that the original DRL agent produces identical outputs for different target nodes given the same graph state, confirming that it cannot condition its policy on the target.

2. **Target-aware architecture.** We propose a novel DRL architecture that explicitly encodes the identity of the target node as an additional input feature. The graph encoder is replaced by Graph Attention Networks v2 (GATv2) [4], which provide a more expressive, query-conditioned attention mechanism compared to the original GNN. Node features combine static indicators derived from the original graph with dynamically updated structural signals that reflect topological changes as rewiring proceeds.

3. **Dual-Agent Architecture.** We extend the single-agent system into a Dual-Agent Architecture that maintains two specialized agents, one for edge addition and one for edge deletion, with separate policy networks and learning dynamics. We show that this specialization leads to improved performance and learning stability.

4. **Policy-Guided Subgraphing.** To make the framework tractable on large graphs, we introduce a Policy-Guided Subgraphing heuristic that restricts the action space to a relevant neighborhood around the target node, reducing computational cost while preserving the quality of the learned policy.

5. **Extensive experimental evaluation.** We evaluate all proposed contributions on the standard benchmark datasets used in the CMH literature, comparing against the original DRL agent and two gradient-based methods introduced in Silvestri, Tolomei, and Bernini [23].

Organization

The remainder of this thesis is organized as follows. Chapter 2 provides the necessary background on graphs, community detection, reinforcement learning, and graph neural networks. Chapter 3 gives a formal problem statement. Chapter 4 reviews related work. Chapter 5 describes the proposed architecture. Chapter 6 presents the experimental evaluation. Chapter 7 concludes with a discussion and directions for future work. Appendices 7.3–7.3 contain supplementary material.

2. BACKGROUND

This chapter introduces the key concepts required to understand the rest of the thesis. We cover the fundamentals of graphs and community detection (Section 2.1), provide a formal definition of Community Membership Hiding as background context (Section 2.3), review the relevant machinery of Reinforcement Learning (Section 2.4), describe Graph Neural Networks and the GATv2 model (Section 2.5), and present the node2vec algorithm for node representation learning (Section 2.6).

2.1 Graphs and Community Structure

2.1.1 Basic Definitions

A *graph* $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ consists of a set of nodes (vertices) $\mathcal{V}$ and a set of edges $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$. We denote $|\mathcal{V}| = N$ and $|\mathcal{E}| = M$. In an undirected graph, $(u, v) \in \mathcal{E}$ implies $(v, u) \in \mathcal{E}$; in a directed graph, these are distinct edges. Most of the datasets in this work are undirected.

The *adjacency matrix* $A \in \{0, 1\}^{N \times N}$ has $A_{ij} = 1$ if $(i, j) \in \mathcal{E}$ and $A_{ij} = 0$ otherwise. For undirected graphs A is symmetric. The *degree* of a node v, denoted $\deg(v)$, equals the number of edges incident to v, i.e., $\deg(v) = \sum_j A_{vj}$. The mean degree of the graph is $\mu = 2M/N$. The *neighborhood* of v is $\mathcal{N}(v) = \{w : (v, w) \in \mathcal{E}\}$.

A *path* of length ℓ from u to v is a sequence of distinct nodes $u = w_0, w_1, \ldots, w_\ell = v$ such that $(w_{i-1}, w_i) \in \mathcal{E}$ for all i. The *shortest-path distance* $d(u, v)$ is the minimum length among all paths from u to v, and the *diameter* of $\mathcal{G}$ is $\max_{u,v} d(u, v)$.

Definition 2.1 (Connected graph)**.** *A graph is* connected *if there exists a path between every pair of nodes. A maximal connected subgraph is called a* connected component*.*

2.1.2 Node Centrality Measures

Centrality measures quantify the structural importance of a node within the graph. Two measures are particularly relevant to this work.

Degree centrality. The simplest centrality measure is normalized degree: $C_D(v) = \deg(v)/(N-1)$. Nodes with high degree centrality have many direct connections and often act as hubs.

Betweenness centrality. Introduced by Freeman [10], betweenness centrality measures how often a node lies on shortest paths between other node pairs:

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}},$$

where σ_{st} is the total number of shortest paths from s to t and $\sigma_{st}(v)$ is the number of those paths passing through v. Nodes with high betweenness are critical bridges between parts of the network.

Clustering coefficient. The *local clustering coefficient* of a node v measures the density of edges among its neighbors:

$$C_C(v) = \frac{|\{(u, w) \in \mathcal{E} : u, w \in \mathcal{N}(v)\}|}{\deg(v)(\deg(v) - 1)/2}.$$

A high clustering coefficient indicates that the neighborhood of v is itself densely connected, a hallmark of community membership.

2.1.3 Graph Models

Real-world networks often share structural properties not captured by classical random graph models. Two such properties are particularly relevant to community detection.

Small-world property. Watts and Strogatz [29] showed that many real-world networks have short average path lengths (logarithmic in N) despite high clustering coefficients. This "small-world" property distinguishes social and biological networks from Erdős-Rényi random graphs.

Scale-free degree distribution. Barabási and Albert [1] observed that degree distributions in many real-world networks follow a power law: $P(\deg = k) \propto k^{-\gamma}$. This implies the existence of highly connected hubs. The degree sequence affects how robust a network is to targeted vs. random node removal, and has direct implications for the difficulty of hiding a high-degree node.

2.1.4 Community Structure

Definition 2.2 (Community structure)**.** *A graph $\mathcal{G}$ exhibits* community structure *if its nodes can be partitioned into groups $\mathcal{C} = \{\mathcal{C}_1, \mathcal{C}_2, \ldots, \mathcal{C}_k\}$ such that the density of edges within each group is significantly higher than the density of edges between different groups.*

Intuitively, communities correspond to densely connected subgraphs (clusters) that are only loosely coupled to one another. A widely used quantitative measure of community quality is *modularity* [19]:

$$Q = \frac{1}{2M} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2M} \right] \delta(c_i, c_j),$$

where k_i is the degree of node i and $\delta(c_i, c_j)$ is 1 if nodes i and j belong to the same community. A value of Q close to 1 indicates strong community structure, while $Q \approx 0$ indicates no more within-community edges than expected under a random null model.

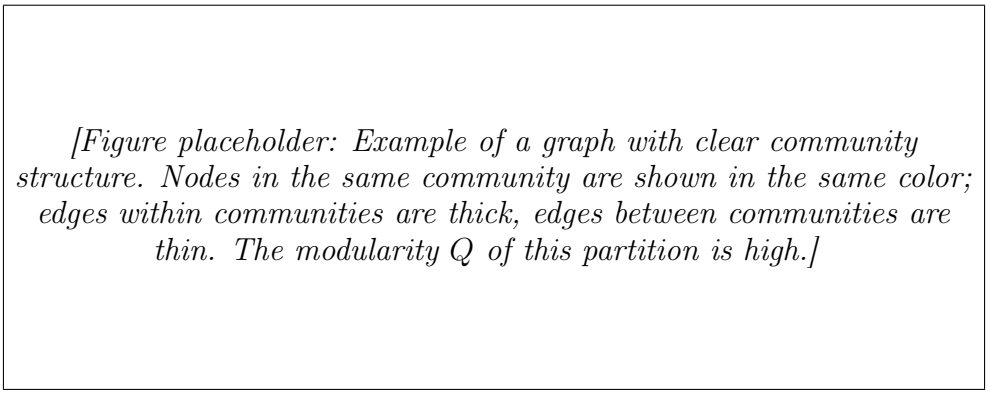

Fig. 2.1: Example of a graph with clear community structure. Nodes in the same community are shown in the same color; edges within communities are thick, edges between communities are thin. The modularity Q of this partition is high.

2.2 Community Detection Algorithms

Many community detection algorithms have been proposed in the literature [9]. In this thesis, we focus on three widely used methods, all of which are treated as *black boxes* by our agent.

Greedy modularity optimization. Clauset, Newman, and Moore [6] introduced a greedy algorithm that maximizes modularity. The algorithm begins with each node in its own community and iteratively merges pairs of communities that yield the largest increase in Q. The procedure terminates when no merge improves Q. Despite its simplicity, the algorithm is widely used due to its efficiency and interpretability.

Louvain algorithm. Blondel et al. [3] proposed a two-phase greedy heuristic for modularity maximization. In the first phase, each node is moved to the community of a neighbor that yields the largest modularity gain (if any such gain exists). In the second phase, communities are collapsed into super-nodes and the first phase is repeated on the resulting graph. The process continues until no further improvement is possible. The Louvain algorithm is highly efficient and widely adopted for large-scale networks.

Walktrap algorithm. Pons and Latapy [21] introduced a community detection algorithm based on random walks. The key intuition is that short random walks tend to stay within the same community; therefore, nodes that are visited by the same set of short random walks are likely to belong to the same community. The algorithm defines a distance between nodes based on the transition probabilities of a t-step random walk and uses hierarchical clustering to identify communities.

Remark 2.1. *The algorithms above all produce a* hard *partition of the node set: each node belongs to exactly one community. Extensions to* overlapping *community detection [9], where nodes may belong to multiple communities simultaneously, exist but are outside the scope of this work.*

2.3 Community Membership Hiding: Informal Overview

In the Community Membership Hiding setting, an actor (the *hider*) controls a specific target node u in a graph $\mathcal{G}$. The hider can add or remove a limited number of edges adjacent to u, with the goal of causing a given community detection algorithm f to no longer assign u to its original community $\mathcal{C}_i$ after the modifications. The detection algorithm is treated as a black box: the hider can query it but cannot access its internals.

This problem models a privacy-conscious user who wants to prevent a platform from correctly inferring their social group, and who can strategically manage their connections to achieve this goal. A formal treatment is given in Chapter 3.

2.4 Reinforcement Learning

Reinforcement Learning (RL) is a machine learning paradigm in which an *agent* learns to make sequential decisions by interacting with an *environment* in order to maximize a cumulative *reward* signal [26]. The interaction proceeds in discrete time steps: at each step t, the agent observes a state $s_t \in \mathcal{S}$, selects an action $a_t \in \mathcal{A}(s_t)$ according to a policy π, and receives a scalar reward r_t while transitioning to state s_{t+1}.

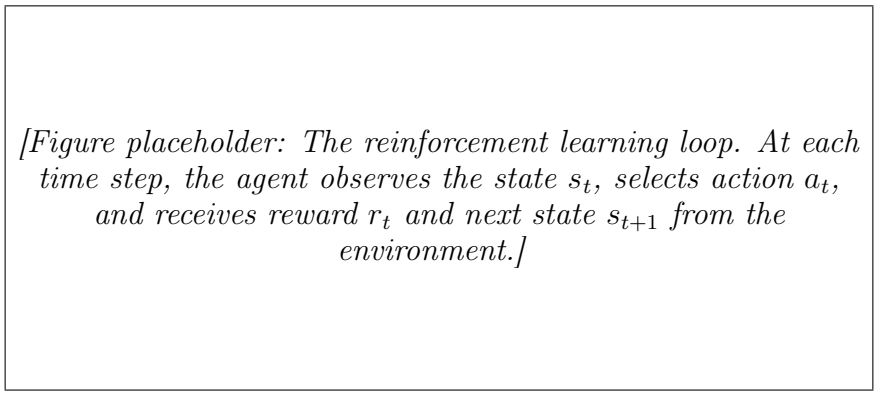

Fig. 2.2: The reinforcement learning loop. At each time step, the agent observes the state s_t, selects action a_t, and receives reward r_t and next state s_{t+1} from the environment.

2.4.1 Markov Decision Processes

The standard formalization is the *Markov Decision Process* (MDP), defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where:

- $\mathcal{S}$ is the *state space*;
- $\mathcal{A}$ is the *action space* (may depend on the current state);
- $P(s' \mid s, a)$ is the *transition probability distribution*;
- $R(s, a)$ is the expected immediate reward;
- $\gamma \in [0, 1]$ is the *discount factor*, which controls how much future rewards are valued relative to immediate ones.

The return from step t is:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}.$$

The objective is to find a policy π^* that maximizes the expected return:

$$\pi^* = \arg\max_{\pi} \mathbb{E}_{\pi} \left[G_t \mid s_t = s \right].$$

2.4.2 Value Functions and the Bellman Equations

A central concept in RL is the *value function*, which estimates the expected return from a given state (or state-action pair) under a fixed policy π.

Definition 2.3 (State-value function)**.** $V^{\pi}(s) = \mathbb{E}_{\pi} \left[G_t \mid s_t = s \right]$

Definition 2.4 (Action-value function)**.** $Q^{\pi}(s,a) = \mathbb{E}_{\pi} \left[G_t \mid s_t = s, a_t = a \right]$

These two functions are related by $V^{\pi}(s) = \sum_a \pi(a \mid s)\, Q^{\pi}(s,a)$. They satisfy the *Bellman equations*:

$$V^{\pi}(s) = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s,a) \left[R(s,a) + \gamma\, V^{\pi}(s') \right], \tag{2.1}$$

$$Q^{\pi}(s,a) = \sum_{s'} P(s' \mid s,a) \left[R(s,a) + \gamma \sum_{a'} \pi(a' \mid s')\, Q^{\pi}(s',a') \right]. \tag{2.2}$$

These recursive equations characterize the value functions of any policy π and form the basis of most RL algorithms.

2.4.3 Policy Gradient Methods

Rather than estimating value functions and deriving a policy from them (value-based methods), *policy gradient* methods directly optimize a parametric policy $\pi_\theta(a \mid s)$. The policy gradient theorem [26, 30] states:

$$\nabla_\theta J(\theta) = \mathbb{E}_{s \sim d^{\pi},\, a \sim \pi_\theta} \left[\nabla_\theta \log \pi_\theta(a \mid s)\, Q^{\pi_\theta}(s,a) \right],$$

where $J(\theta)$ is the expected return and d^{π} is the stationary state distribution under policy π_θ. In practice, $Q^{\pi_\theta}(s,a)$ is replaced by an estimate. Replacing it with the *advantage function*

$$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$$

reduces variance without introducing bias, since subtracting a function of the state alone (the baseline $V^{\pi}(s)$) does not affect the gradient in expectation:

$$\nabla_\theta J(\theta) = \mathbb{E} \left[\nabla_\theta \log \pi_\theta(a \mid s)\, A^{\pi_\theta}(s,a) \right].$$

2.4.4 Actor-Critic Methods and A2C

Actor-Critic architectures maintain two networks:

- The *actor*, representing the policy $\pi_\theta(a \mid s)$, updated to maximize expected return.
- The *critic*, representing the value function $V_\phi(s)$, updated to minimize the squared Bellman error. The critic's output serves as the baseline for variance reduction.

The *Advantage Actor-Critic* (A2C) algorithm [17] is a synchronous variant that collects transitions from multiple workers in parallel, computes advantages using n-step returns, and updates both networks simultaneously.

Algorithm 1 A2C update (one iteration)

Require: Policy π_θ, value network V_ϕ, collected trajectory $(s_0, a_0, r_0, \ldots, s_{T-1}, a_{T-1}, r_{T-1}, s_T)$

1: Compute n-step returns: $R_t \leftarrow \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V_\phi(s_{t+n})$
2: Compute advantages: $A_t \leftarrow R_t - V_\phi(s_t)$
3: Actor loss: $\mathcal{L}_{\text{actor}} \leftarrow -\frac{1}{T} \sum_t \log \pi_\theta(a_t \mid s_t) \cdot A_t$
4: Critic loss: $\mathcal{L}_{\text{critic}} \leftarrow \frac{1}{T} \sum_t \big(R_t - V_\phi(s_t)\big)^2$
5: Entropy bonus: $\mathcal{L}_{\text{ent}} \leftarrow -\frac{1}{T} \sum_t H\big(\pi_\theta(\cdot \mid s_t)\big)$
6: $\mathcal{L} \leftarrow \mathcal{L}_{\text{actor}} + c_v \mathcal{L}_{\text{critic}} + c_e \mathcal{L}_{\text{ent}}$
7: Update θ, ϕ via gradient descent on $\mathcal{L}$

The entropy bonus $\mathcal{L}_{\text{ent}}$ encourages the policy to remain stochastic during training, preventing premature convergence to a suboptimal deterministic policy. The coefficients c_v and c_e are hyperparameters that balance the relative magnitudes of the three loss components.

2.5 Graph Neural Networks

Graph Neural Networks (GNNs) are a family of deep learning models designed to operate on graph-structured data [22]. The core idea is *message passing* [11]: each node aggregates information from its neighbors, transforming the aggregated messages through learnable parameters to produce a new node representation.

2.5.1 The Message Passing Framework

Formally, the ℓ-th layer of a message-passing GNN updates the representation of node v as follows:

$$\mathbf{m}_v^{(\ell)} = \text{AGG}^{(\ell)}\Big(\Big\{\mathbf{h}_u^{(\ell-1)} : u \in \mathcal{N}(v)\Big\}\Big), \tag{2.3}$$

$$\mathbf{h}_v^{(\ell)} = \text{UPDATE}^{(\ell)}\Big(\mathbf{h}_v^{(\ell-1)}, \mathbf{m}_v^{(\ell)}\Big), \tag{2.4}$$

where $\mathbf{h}_v^{(0)} = \mathbf{x}_v$ is the input feature vector of node v. After L layers, the representation $\mathbf{h}_v^{(L)}$ captures information from all nodes within L hops of v. Different GNN architectures differ in how they define AGG and UPDATE.

Graph Convolutional Networks (GCN). The GCN [14] uses a symmetric normalized aggregation:

$$\mathbf{h}_v^{(\ell)} = \sigma\left(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \frac{1}{\sqrt{\hat{d}_v \hat{d}_u}} \mathbf{W}^{(\ell)} \mathbf{h}_u^{(\ell-1)} \right),$$

where $\hat{d}_v = 1 + \deg(v)$ accounts for the self-loop added to each node. The normalization prevents the embeddings of high-degree nodes from dominating the representation.

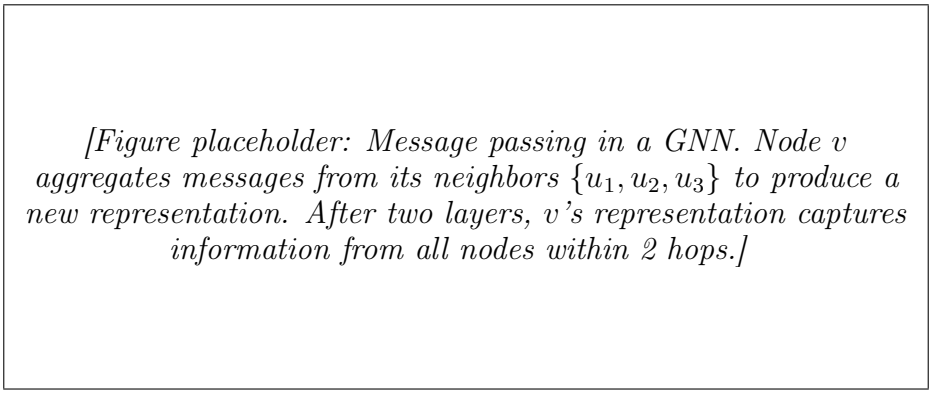

Fig. 2.3: Message passing in a GNN. Node v aggregates messages from its neighbors $\{u_1, u_2, u_3\}$ to produce a new representation. After two layers, v's representation captures information from all nodes within 2 hops.

Graph Isomorphism Networks (GIN). Xu et al. [31] showed that the expressive power of a GNN is upper-bounded by that of the Weisfeiler-Lehman graph isomorphism test, and proposed GIN, which matches this upper bound:

$$\mathbf{h}_v^{(\ell)} = \mathrm{MLP}^{(\ell)}\left((1 + \epsilon^{(\ell)})\, \mathbf{h}_v^{(\ell-1)} + \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(\ell-1)} \right),$$

where ϵ is a learnable scalar. GIN is a stronger baseline for graph classification but is not the model we use; it is presented here for context.

2.5.2 Graph Attention Networks (GAT)

Graph Attention Networks [28] replace uniform or degree-based aggregation with a learnable attention mechanism. The ℓ-th layer computes:

$$\mathbf{h}_v^{(\ell)} = \sigma\left(\sum_{u \in \mathcal{N}(v)} \alpha_{vu}\, \mathbf{W}^{(\ell)} \mathbf{h}_u^{(\ell-1)} \right),$$

where the attention coefficient α_{vu} is:

$$\alpha_{vu} = \frac{\exp\big(\mathrm{LeakyReLU}\big(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_v \,\|\, \mathbf{W}\mathbf{h}_u]\big)\big)}{\sum_{w \in \mathcal{N}(v)} \exp(\mathrm{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_v \,\|\, \mathbf{W}\mathbf{h}_w]))}.$$

Multi-head attention uses K independent attention heads and concatenates (or averages) their outputs, providing more stable learning and richer representations.

2.5.3 GATv2: Dynamic Graph Attention

Brody, Alon, and Yahav [4] identified a fundamental limitation of the original GAT: its attention is *static*, meaning the ranking of neighbors is the same for all query nodes. This occurs because the attention scoring applies the linear transformation *before* the non-linearity, making neighbor rankings independent of the query node's features.

GATv2 corrects this by reordering the operations:

$$e(\mathbf{h}_v, \mathbf{h}_u) = \mathbf{a}^\top \mathrm{LeakyReLU}(\mathbf{W}_1\mathbf{h}_v + \mathbf{W}_2\mathbf{h}_u)\,,$$

which makes the scoring function a universal approximator over pairs of node representations. The resulting attention is *dynamic*: the ranking of neighbors can differ for every query node, making GATv2 strictly more expressive than GAT. This property is particularly desirable in our setting, where the relevance of each neighbor to the hiding objective should depend on the target's own features.

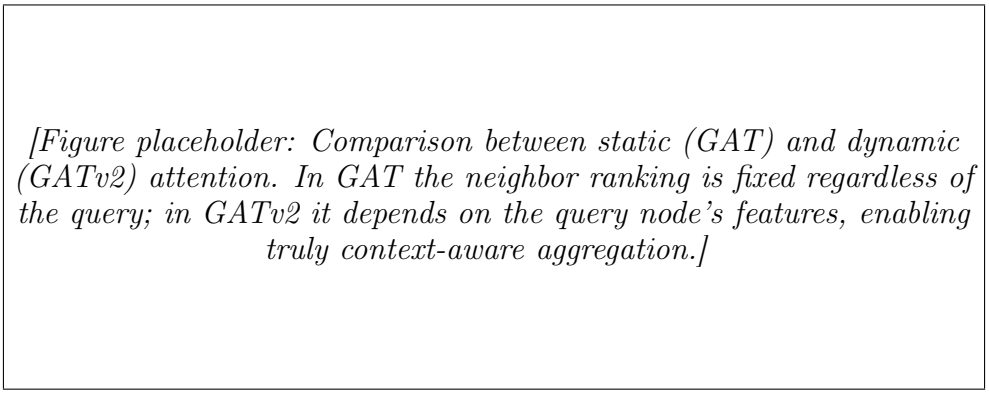

Fig. 2.4: Comparison between static (GAT) and dynamic (GATv2) attention. In GAT the neighbor ranking is fixed regardless of the query; in GATv2 it depends on the query node's features, enabling truly context-aware aggregation.

Proposition 2.1. *Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a graph and let $u, v \in \mathcal{V}$ be two different nodes. In the original GAT [28], the attention score $e(h_u, h_w)$ for any neighbor w is independent of the query node u; thus the attention distribution over $\mathcal{N}(u)$ is identical to that over $\mathcal{N}(v)$ when $h_u = h_v$. In GATv2 this is not the case in general, since the concatenation $[W_1 h_u \| W_2 h_w]$ involves h_u non-linearly.*

2.6 The node2vec Algorithm

In many real-world graphs, nodes have no natural feature vectors. *node2vec* [12] addresses this by learning a d-dimensional embedding for each node via a procedure analogous to word2vec [16]. The algorithm has two stages.

Stage 1 — Biased random walks. Starting from each node v, the algorithm generates a sequence of nodes by performing a *biased second-order random walk*. The transition probability from v to a neighbor x depends on the previously visited node t:

$$P(c_i = x \mid c_{i-1} = v,\, c_{i-2} = t) \propto \pi_{vx} \cdot w_{vx},$$

where w_{vx} is the edge weight and π_{vx} is a bias factor governed by two parameters:

- p (*return parameter*): controls the likelihood of revisiting t. Low p encourages a depth-first (DFS) search, exploring farther from the source.
- q (*in-out parameter*): controls the exploration of distant nodes vs. nearby nodes. Low q encourages a breadth-first (BFS) search, staying local.

By tuning p and q, node2vec interpolates between capturing *homophily* (nodes in the same community are embedded close together) and *structural equivalence* (nodes playing similar structural roles are embedded close together).

Stage 2 — Skip-gram training. The generated walk sequences are treated as "sentences" over a "vocabulary" of node identifiers. A skip-gram model [16] with negative sampling is trained to predict context nodes given a center node, maximizing:

$$\sum_{v \in \mathcal{V}} \log P(\mathcal{N}_S(v) \mid v)\,,$$

where $\mathcal{N}_S(v)$ is the multiset of nodes co-occurring with v across the sampled walks. The result is an embedding matrix $\mathbf{Z} \in \mathbb{R}^{N \times d}$ where geometrically close embeddings correspond to nodes that frequently co-occur in walks.

Limitation in the CMH context. In the original CMH architecture [2], node2vec embeddings are computed *once* on the original graph $\mathcal{G}$ and reused throughout the entire episode. As the rewiring process modifies the graph, the embeddings become increasingly stale, and computing node2vec requires a full random-walk preprocessing step that is expensive for large graphs. One of our proposed improvements is to replace these static embeddings with dynamic structural features recomputed at each step.

[Figure placeholder: Illustration of node2vec biased random walks. With low p and high q (DFS-like behavior, top), the walk explores farther from the source, capturing structural roles. With high p and low q (BFS-like, bottom), the walk stays local, capturing community membership.]

Fig. 2.5: Illustration of node2vec biased random walks. With low p and high q (DFS-like behavior, top), the walk explores farther from the source, capturing structural roles. With high p and low q (BFS-like, bottom), the walk stays local, capturing community membership.

3. PROBLEM FORMULATION

In this chapter we give a formal definition of the Community Membership Hiding problem. Section 3.1 introduces the deception condition, the edge-modification budget, and the admissible action set. Sections 3.2–3.3 define the graph-level and community-level similarity measures used throughout. Section 3.4 presents the evaluation metrics (SR, NMI, F1), and Section 3.5 closes with the MDP formulation adopted by the DRL agent.

3.1 Formal Definition

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be an undirected, unweighted graph and let f be a community detection algorithm that, given $\mathcal{G}$, produces a partition $\mathcal{C} = f(\mathcal{G}) = \{\mathcal{C}_1, C_2, \ldots, C_k\}$ of $\mathcal{V}$ into k non-overlapping communities.

Given a target node $u \in \mathcal{V}$ with original community assignment $\mathcal{C}_i = f_u(\mathcal{G})$ (the community to which u belongs in $\mathcal{G}$), the goal of CMH is to find a modified graph $\mathcal{G}' = (\mathcal{V}, \mathcal{E}')$ such that the following *deception condition* is satisfied:

Definition 3.1 (Deception condition)**.** *Let $\mathcal{C}'_i = f_u(\mathcal{G}')$ denote the community assigned to u in $\mathcal{G}'$. The deception condition is:*

$$\mathrm{sim}(\mathcal{C}_i \setminus \{u\},\ \mathcal{C}'_i \setminus \{u\}) \leq \tau,$$

where sim *is a community similarity function and $\tau \in [0, 1]$ is the* deception threshold*. A value of $\tau = 0$ requires $\mathcal{C}_i$ and $\mathcal{C}'_i$ to be completely disjoint (hard deception), while $\tau = 1$ permits them to be identical (trivially satisfied).*

The node u is excluded from both communities in the similarity computation, because u trivially appears in both communities before and after the rewiring; the intent is to measure whether u has migrated to a genuinely different community.

Following Bernini, Silvestri, and Tolomei [2], we use the *Sørensen-Dice Coefficient* (DSC) as sim [24]:

$$\mathrm{DSC}(A, B) = \frac{2|A \cap B|}{|A| + |B|}.$$

DSC measures the proportion of shared nodes between two communities: it equals 0 when the communities are disjoint and 1 when they are identical.

Definition 3.2 (Budget constraint)**.** *The number of edge modifications is bounded by a* budget $\beta = \lfloor \beta_{\mathrm{mul}} \cdot \mu \rfloor$*, where $\mu = 2M/N$ is the mean degree and $\beta_{\mathrm{mul}} \in \{1/2, 1, 2\}$ is a multiplier. Each rewiring step constitutes a single modification.*

Definition 3.3 (Admissible actions)**.** *To align with the hiding objective, the action space is restricted as follows. Let $\mathcal{C}_i = f_u(\mathcal{G})$ be the current community of u:*

- ***Edge removal****: the agent may remove an edge (u, v) only if $v \in \mathcal{C}_i$ (i.e., v is an* intra-community *neighbor of u).*
- ***Edge addition****: the agent may add an edge (u, v) only if $v \notin \mathcal{C}_i$ (i.e., v is an* inter-community *candidate).*

The motivation for restricting the action space is straightforward: removing inter-community edges would further isolate u within $\mathcal{C}_i$, and adding intra-community edges would reinforce u's membership. Both are thus dominated strategies and excluded from consideration.

Remark 3.1. *Note that the admissibility of an action depends on the* current *community assignment $f_u(\mathcal{G}_t)$, which may change during an episode as rewiring progresses. In practice, we recompute $f(\mathcal{G}_t)$ after each step, so the action set is always up-to-date.*

3.2 Graph Similarity

Beyond the community-level deception condition, we also measure the global structural distortion introduced by the rewiring. Following the original paper, we use the *Jaccard similarity* of the edge sets [13]:

$$\mathrm{JAC}(\mathcal{G}, \mathcal{G}') = \frac{|\mathcal{E} \cap \mathcal{E}'|}{|\mathcal{E} \cup \mathcal{E}'|}.$$

A high Jaccard similarity indicates that the modified graph closely resembles the original, i.e., the structural cost of hiding is low.

3.3 Normalized Mutual Information

To quantify the impact of the rewiring on the *community structure* as a whole, we use the Normalized Mutual Information (NMI) [25] between the original partition $\mathcal{C} = f(\mathcal{G})$ and the modified partition $\mathcal{C}' = f(\mathcal{G}')$:

$$\mathrm{NMI}(\mathcal{C}, \mathcal{C}') = \frac{2\, I(\mathcal{C}\,;\, \mathcal{C}')}{H(\mathcal{C}) + H(\mathcal{C}')},$$

where I denotes mutual information and H entropy over the community assignments. NMI takes values in $[0, 1]$, with 1 indicating identical partitions and 0 indicating no shared information. A high NMI after hiding means the global community structure has been well preserved.

3.4 Evaluation Metrics

We evaluate methods using three metrics:

Success Rate (SR). The fraction of test instances in which the deception condition (Definition 3.1) is satisfied:

$$\mathrm{SR} = \frac{\#\,\text{successful runs}}{\#\,\text{total runs}}.$$

NMI. The average NMI between $f(\mathcal{G})$ and $f(\mathcal{G}')$ across all test instances. Higher NMI indicates less structural distortion.

F1 score. Since SR and NMI are competing objectives (higher SR typically requires more aggressive rewiring, which lowers NMI), we report their harmonic mean:

$$F_1 = \frac{2 \cdot \mathrm{SR} \cdot \mathrm{NMI}}{\mathrm{SR} + \mathrm{NMI}}.$$

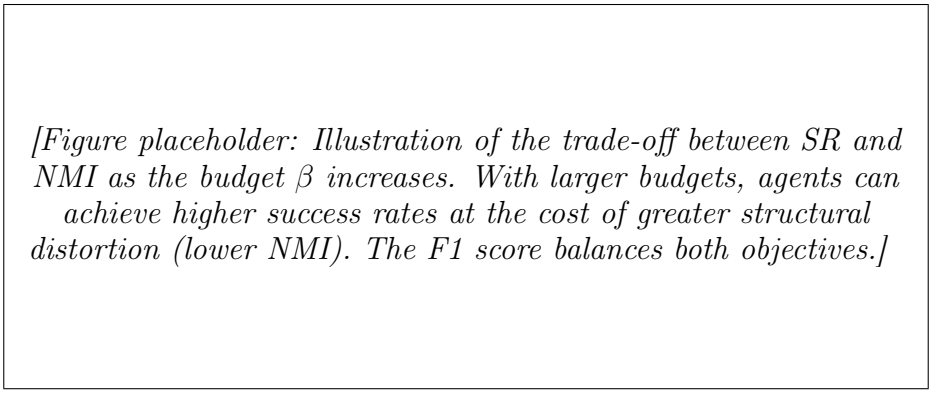

Fig. 3.1: Illustration of the trade-off between SR and NMI as the budget β increases. With larger budgets, agents can achieve higher success rates at the cost of greater structural distortion (lower NMI). The F1 score balances both objectives.

3.5 MDP Formulation

We formalize the CMH problem as an MDP $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

- **State** s_t: the current graph $\mathcal{G}_t = (\mathcal{V}, \mathcal{E}_t)$ together with the current community partition $\mathcal{C}_t = f(\mathcal{G}_t)$ and the identity of the target node u.
- **Action** a_t: a node v from the admissible action set (Definition 3.3). The environment interprets this as either adding or removing the edge (u, v) depending on whether $v \notin \mathcal{E}_t$ or $v \in \mathcal{E}_t$.
- **Transition**: deterministic given s_t and a_t; the graph is updated accordingly and the community partition is recomputed.
- **Reward** r_t: a shaped signal described in Section 5.6.
- **Horizon**: the episode terminates when the deception condition is met or the budget β is exhausted.

4. RELATED WORK

This chapter surveys the work most directly related to our contributions. Section 4.1 describes the original DRL-based CMH agent of Bernini, Silvestri, and Tolomei [2], which serves as our primary baseline. Section 4.2 presents the gradient-based methods of Silvestri, Tolomei, and Bernini [23], which constitute a second set of strong baselines. Sections 4.3–4.5 situate our work in the broader context of adversarial attacks on graphs, privacy in social networks, and reinforcement learning for combinatorial optimization on graphs.

4.1 The Original CMH Agent (Bernini et al., 2024)

Bernini, Silvestri, and Tolomei [2] were the first to formulate Community Membership Hiding as an MDP and solve it with DRL. Their architecture, which serves as our primary baseline, consists of:

1. A **graph encoder** based on a GNN that produces a representation of the graph state. Node features are either random vectors or node2vec embeddings computed on the original graph.

2. An **Actor-Critic network** (A2C) that, given the encoded graph, outputs a probability distribution over candidate nodes for the next rewiring step and a scalar value estimate.

The agent is trained on the `words` dataset using the `greedy` community detection algorithm, and is evaluated for transferability to other datasets and algorithms.

The original agent achieves success rates significantly above the random and degree-based baselines in standard evaluation scenarios. However, it degrades substantially on larger graphs; as we establish in Chapter 5, it also suffers from a fundamental architectural limitation regarding the role of the target node.

4.2 The Right to Hide (Silvestri et al., 2025)

Silvestri, Tolomei, and Bernini [23] approached CMH from a different angle, framing it as a continuous optimization problem. They define a differentiable relaxation of the deception loss and apply projected gradient descent, yielding two variants: ∇-CMH and ∇-CMH (projected). These gradient-based methods are computationally efficient and achieve competitive hiding rates with a well-controlled structural footprint. We use both as strong baselines in our evaluation.

4.3 Adversarial Attacks on Graphs

CMH belongs to a broader literature on *adversarial attacks on graphs*, where the objective is to modify graph structure so as to fool a downstream algorithm. The most studied setting is attacking GNN-based node classifiers: Zügner, Akbarnejad, and Günnemann [32] introduced Nettack, a perturbation method for targeted evasion attacks on GNNs, while

Dai et al. [7] used reinforcement learning (Q-learning) to attack both node classification and graph classification models.

Community evasion was studied in the Q-attack framework by Chen et al. [5], which uses a genetic algorithm combined with Q-learning to attack community detection algorithms. Unlike CMH, which focuses on a single target node, Q-attack targets the global community structure.

4.4 Privacy in Social Networks

The privacy motivation for CMH connects to a large body of work on network anonymization and deanonymization. Narayanan and Shmatikov [18] showed that social networks can be deanonymized even when node identifiers are removed, by exploiting structural patterns. Fire, Goldschmidt, and Elovici [8] survey the broader landscape of threats and defenses for online social networks, including inference attacks that exploit community membership.

These works highlight the practical importance of methods that allow individuals to actively protect their structural privacy, rather than relying solely on anonymization by the platform.

4.5 RL for Combinatorial Optimization

Solving combinatorial optimization problems with RL is an active research area [15]. CMH is a combinatorial problem (finding an optimal subset of edge modifications) with a discrete action space. RL approaches are particularly attractive here because the objective is non-differentiable (it requires running a community detection algorithm, which is not generally differentiable with respect to the graph adjacency). Tang et al. [27] survey RL methods specifically designed for graph-structured problems.

5. PROPOSED ARCHITECTURE

In this chapter we first analyze the original DRL agent and identify its key limitations (Section 5.1). We then describe our proposed improvements step by step: target-aware node features (Section 5.2), the GATv2-based graph encoder (Section 5.3), the Dual-Agent Architecture (Section 5.4), and Policy-Guided Subgraphing (Section 5.5). Section 5.6 describes the training procedure and reward structure.

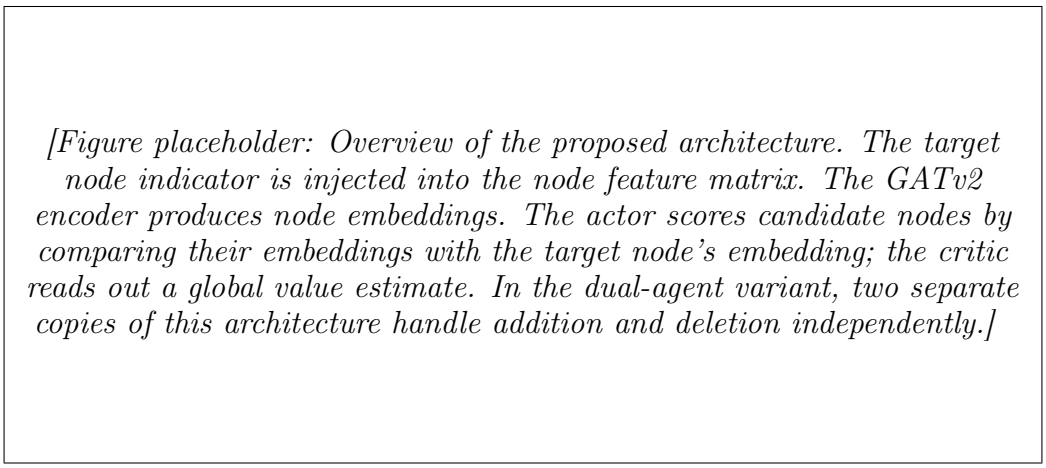

Fig. 5.1: Overview of the proposed architecture. The target node indicator is injected into the node feature matrix. The GATv2 encoder produces node embeddings. The actor scores candidate nodes by comparing their embeddings with the target node's embedding; the critic reads out a global value estimate. In the dual-agent variant, two separate copies of this architecture handle addition and deletion independently.

5.1 Analysis of the Original Agent

5.1.1 The Target-Node Blindness Problem

A central observation motivating our work is the following. In the original architecture [2], the graph encoder receives the full graph state $\mathcal{G}_t$ as input and produces a vector representation for every node. The actor network then outputs a probability distribution over all candidate nodes, and the environment applies the selected action *relative to the target node* u (e.g., removes the edge between u and the selected intra-community neighbor).

The critical issue is that nowhere in this pipeline is the identity of u communicated to the network. The encoder and actor receive no signal about *who* the target is. To verify this empirically, we fixed all random seeds and ran the original agent on the same graph with different target nodes. The actor's output probabilities and the critic's value estimate were identical across all target choices in the first step of every episode, confirming that the model is entirely unaware of u.

Proposition 5.1 (Target blindness of the original agent)**.** *Let* $\mathbf{X} \in \mathbb{R}^{N \times d}$ *be the node feature matrix used by the original agent. If the features of all nodes are drawn independently of* u *(as in the original implementation, where features are random vectors or node2vec*

embeddings computed on the unmodified graph), then the output distribution of the actor is invariant to the choice of target node u. Hence the agent cannot implement a target-specific policy.

As a result, the agent learns a global rewiring strategy that tends to increase overall community mixing. While this can accidentally displace u from its community, it is neither efficient nor reliable, especially when the budget β is small.

5.1.2 Static Node Embeddings

The original architecture uses node2vec embeddings computed *once* on the original graph $\mathcal{G}$ and reused throughout the entire episode. As the rewiring process modifies the graph, the embeddings become increasingly stale and fail to reflect the current topology, and computing node2vec requires a separate preprocessing step that is itself expensive for large graphs.

5.1.3 Undifferentiated Action Space

The original agent handles both edge addition and edge removal with the same policy network, without distinguishing between the two types of actions. We hypothesize that these two tasks have fundamentally different structure: removing intra-community edges requires identifying which neighbors contribute most to u's community membership, while adding inter-community edges requires identifying which external nodes are most useful for attracting u to a different community.

5.2 Target-Aware Node Feature Construction

Our first modification addresses the target-node blindness problem. For each node $v \in \mathcal{V}$, we construct a feature vector $\mathbf{x}_v \in \mathbb{R}^d$ that explicitly encodes whether v is the target node:

$$\mathbf{x}_v = \Big[f_{\text{struct}}(v) \;\Big|\; \mathbf{1}[v = u] \Big],$$

where $f_{\text{struct}}(v)$ is a vector of structural features and $\mathbf{1}[v = u]$ is a binary indicator that is 1 only for the target node u.

The structural features $f_{\text{struct}}(v)$ combine:

- **Static features** computed on the original graph $\mathcal{G}_0$ (degree, clustering coefficient, betweenness centrality, community label) and held fixed throughout the episode. These capture the initial structural role of each node.
- **Dynamic features** recomputed at each step on the current graph $\mathcal{G}_t$ (current degree, current community label, current number of intra-community and inter-community neighbors). These capture the real-time effect of previous rewiring actions.

By including $\mathbf{1}[v = u]$ in the feature vector, the GATv2 encoder can compute attention weights that depend explicitly on whether a node is the target. This enables the model to give special treatment to u's neighborhood and to aggregate information relevant to the hiding objective.

5.3 GATv2-Based Graph Encoder

We replace the original GNN encoder with a multi-layer GATv2 [4] network. As discussed in Section 2.5, GATv2 provides dynamic attention: the relative importance of each neighbor can differ for every query node, which is essential when the target node u needs to focus on a specific subset of its neighbors.

The GATv2 encoder takes as input the current graph $\mathcal{G}_t$ with node features $\mathbf{X}_t \in \mathbb{R}^{N \times d}$ and produces a node embedding matrix $\mathbf{H} \in \mathbb{R}^{N \times d'}$. The actor and critic networks operate on these embeddings.

Actor. The actor scores each candidate node v in the action set $\mathcal{A}(s_t)$:

$$\text{score}(v) = \text{MLP}_{\text{actor}}([\mathbf{h}_u \,\|\, \mathbf{h}_v]) \,,$$

where $\mathbf{h}_u$ and $\mathbf{h}_v$ are the embeddings of the target node and candidate v respectively, and $\|\,\|$ denotes concatenation. This formulation ensures that the score of each candidate is conditioned on the target node's embedding, directly linking the action selection to the target's current context.

The action probability distribution is obtained by applying a softmax over all candidates in $\mathcal{A}(s_t)$.

Critic. The critic estimates the state value:

$$V(s_t) = \text{MLP}_{\text{critic}}(\text{READOUT}(\mathbf{H})) \,,$$

where READOUT is a mean pooling over all node embeddings, followed by a linear transformation.

5.4 Dual-Agent Architecture

The Dual-Agent Architecture maintains two independent Actor-Critic systems:

- $\text{Agent}_{\text{add}}$: handles edge *addition* actions (selecting inter-community candidates to connect to u).
- $\text{Agent}_{\text{del}}$: handles edge *removal* actions (selecting intra-community neighbors of u whose edge should be removed).

At each step, the environment determines whether the next action should be an addition or a deletion (based on availability and the current budget), and dispatches to the appropriate agent. Each agent has its own GATv2 encoder, actor network, and critic network, with parameters trained independently.

[Figure placeholder: Dual-Agent Architecture. At each step, the dispatcher routes the decision to the addition agent $\text{Agent}_{\text{add}}$ *or the deletion agent* $\text{Agent}_{\text{del}}$*, each with its own encoder, actor, and critic.]*

Fig. 5.2: Dual-Agent Architecture. At each step, the dispatcher routes the decision to the addition agent $\text{Agent}_{\text{add}}$ or the deletion agent $\text{Agent}_{\text{del}}$, each with its own encoder, actor, and critic.

The rationale is that edge addition and edge deletion are structurally different tasks. The deletion agent needs to identify which intra-community connections most strongly anchor u to its community; the addition agent needs to identify which external nodes would most effectively attract u to a different community. Separating these tasks into specialized networks allows each agent to develop representations tailored to its subtask.

5.5 Policy-Guided Subgraphing

On large graphs with thousands or tens of thousands of nodes, the full graph cannot be efficiently processed by a GNN in each step of every episode. Policy-Guided Subgraphing addresses this by restricting the graph encoder to a subgraph $\hat{\mathcal{G}}_t$ centered on the target node:

$$\hat{\mathcal{G}}_t = \mathcal{G}_t\left[\bigcup_{k=0}^{K} \mathcal{N}^k(u)\right],$$

where $\mathcal{N}^k(u)$ denotes the set of nodes reachable from u in at most k hops in $\mathcal{G}_t$. The subgraph retains all edges within this neighborhood. In our experiments, $K = 2$ (the 2-hop ego-subgraph of u) provides a good trade-off between representational richness and computational tractability.

The action space is further restricted to nodes within $\hat{\mathcal{G}}_t$: inter-community candidates are drawn from the 2-hop neighborhood of u, and intra-community neighbors are a subset of u's direct neighbors. This heuristic is *policy-guided* in the sense that the agent's decisions are made with respect to the local context of u rather than the global graph, which is both more efficient and more aligned with the semantics of the hiding task.

5.6 Training Procedure

We train the agent (or agents, in the dual-agent setting) on the `words` dataset using the `greedy` community detection algorithm, following the same setting as the original work. Community selection method 1 (random community selection) is used during training to expose the agent to diverse node types and community structures, avoiding overfitting to a fixed node or community.

We use the A2C update rule from Algorithm 1, with losses:

$$\begin{aligned}\mathcal{L}_{\text{actor}} &= -\log \pi_\theta(a_t \mid s_t)\, A_t,\\ \mathcal{L}_{\text{critic}} &= \left(R_t - V_\phi(s_t)\right)^2,\\ \mathcal{L}_{\text{total}} &= \mathcal{L}_{\text{actor}} + c_v\, \mathcal{L}_{\text{critic}} - c_e\, H(\pi_\theta(\cdot \mid s_t)),\end{aligned}$$

where $A_t = R_t - V_\phi(s_t)$ is the estimated advantage, H is the entropy regularization term that encourages exploration, and c_v, c_e are coefficients.

The reward at each step is a combination of a progress reward (how much the deception condition has improved) and a penalty that discourages excessive structural perturbation:

$$r_t = \alpha\, \Delta_{\text{sim}}(t) - (1-\alpha)\, \Delta_{\text{NMI}}(t),$$

where Δ_{sim} is the reduction in the DSC similarity (progress towards hiding) and Δ_{NMI} is the increase in structural distortion. The parameter α controls the trade-off between hiding effectiveness and structural preservation.

[Figure placeholder: Training curves (loss and success rate on the training set) for the proposed architecture versus the original agent, as a function of training episodes.]

Fig. 5.3: Training curves (loss and success rate on the training set) for the proposed architecture versus the original agent, as a function of training episodes.

6. EXPERIMENTAL EVALUATION

This chapter evaluates all proposed contributions against the baselines described in Chapter 4. Section 6.1 describes the experimental environment. Section 6.2 introduces the benchmark datasets and Section 6.3 defines the baselines. Section 6.4 specifies the evaluation protocol. Section 6.5 reports the results, organized as follows: comparison with the original agent (Section 6.5.1), comparison with gradient-based methods (Section 6.5.2), an ablation study (Section 6.5.3), a scalability analysis (Section 6.5.4), and evaluation in the asymmetric setting (Section 6.5.5).

6.1 Setup

All experiments were run on an AWS EC2 `g4dn.xlarge` instance (4 vCPUs at 2.5 GHz, 16 GiB RAM, NVIDIA T4 GPU) running Ubuntu 24.04 with the Deep Learning Base OSS Nvidia Driver AMI. The training algorithm is A2C with the Adam optimizer and a learning rate of 7×10^{-4}. We train for 10,000 episodes on `words` and report results averaged over 100 test runs per configuration, with fixed random seeds for reproducibility.

6.2 Datasets

We use the five datasets from the original CMH benchmark:

Tab. 6.1: Dataset statistics.

Dataset	Nodes	Edges	Avg. degree
`words`	—	—	—
`kar`	—	—	—
`vote`	—	—	—
`pow`	—	—	—
`fb-75`	—	—	—

The `words` dataset is used exclusively for training. The remaining four datasets are used for evaluation, probing the agent's ability to generalize to unseen graphs and, in the asymmetric setting, to unseen community detection algorithms.

6.3 Baselines

We compare against the following baselines:

1. **Random**: selects a random node from the admissible action set.
2. **Degree**: selects the admissible node with the highest degree.
3. **Betweenness**: selects the admissible node with the highest betweenness centrality.
4. **Roam**: a heuristic that reduces the centrality of the target node within its community.

5. **Greedy**: a relaxed heuristic that greedily selects the most promising rewiring step based on node degree.

6. **Original DRL agent** [2]: the A2C agent from the original paper.

7. **∇-CMH** and **∇-CMH (projected)** [23]: gradient-based methods.

6.4 Evaluation Protocol

We evaluate each method on all combinations of $\tau \in \{0.3, 0.5, 0.8\}$ and $\beta_{\text{mul}} \in \{0.5, 1, 2\}$. For each combination and dataset, we run 100 test episodes. In each episode, the target node u is sampled using community selection method 2 (sampling from communities at the 0.2, 0.5, and 0.8 quantiles of community size), providing a balanced evaluation across community sizes.

We evaluate both the *symmetric* setting (the agent is tested with the same detection algorithm used for training, `greedy`) and the *asymmetric* setting (tested with `louvain` and `walktrap`), which assesses the transferability of the learned policy to unseen detection algorithms.

6.5 Results

[Results to be finalized. Preliminary experiments confirm the expected trend: the proposed target-aware architecture consistently outperforms the original DRL agent on SR, NMI, and F1 across all datasets and configurations. Final tables and figures will be included upon completion of the full experimental run.]

6.5.1 Comparison with the Original Agent

[Figure placeholder: SR, NMI, and F1 comparison between the original DRL agent and the proposed target-aware architecture across all test datasets and budget multipliers ($\beta_{\text{mul}} \in \{0.5, 1, 2\}$), symmetric setting ($\tau = 0.5$).]

Fig. 6.1: SR, NMI, and F1 comparison between the original DRL agent and the proposed target-aware architecture across all test datasets and budget multipliers ($\beta_{\text{mul}} \in \{0.5, 1, 2\}$), symmetric setting ($\tau = 0.5$).

6.5.2 Comparison with Gradient-Based Methods

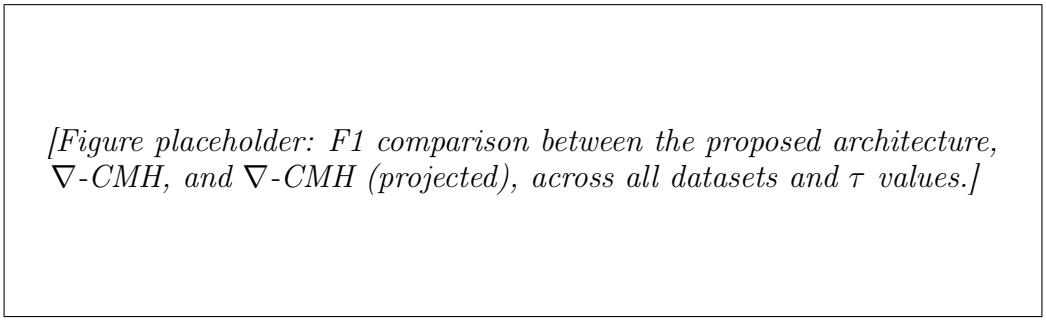

Fig. 6.2: F1 comparison between the proposed architecture, ∇-CMH, and ∇-CMH (projected), across all datasets and τ values.

6.5.3 Ablation Study

To isolate the contribution of each component, we conduct an ablation study comparing:

1. **Original**: the original DRL agent [2].
2. + **Target feature**: original architecture with the target-node indicator added.
3. + **GATv2**: target-aware architecture with GATv2 encoder.
4. + **Dual agents**: full proposed architecture with dual-agent specialization.

Tab. 6.2: Ablation study results (SR / NMI / F1) on `kar` and `vote`, symmetric setting, $\tau = 0.5$, $\beta_{\text{mul}} = 1$.

Method	`kar`			`vote`		
	SR	NMI	F1	SR	NMI	F1
Original	—	—	—	—	—	—
+ Target feature	—	—	—	—	—	—
+ GATv2	—	—	—	—	—	—
+ Dual agents (full)	—	—	—	—	—	—

6.5.4 Scalability

[Figure placeholder: Runtime and F1 comparison on `fb-75` with and without Policy-Guided Subgraphing ($K = 2$). The x-axis shows the graph size (number of nodes processed per GNN forward pass); the y-axis shows wall-clock time per episode.]

Fig. 6.3: Runtime and F1 comparison on `fb-75` with and without Policy-Guided Subgraphing ($K = 2$). The x-axis shows the graph size (number of nodes processed per GNN forward pass); the y-axis shows wall-clock time per episode.

[Results on `fb-75` with and without Policy-Guided Subgraphing to be included.]

6.5.5 Asymmetric Evaluation

[Figure placeholder: Asymmetric evaluation: SR and NMI when the agent trained on `greedy` is tested on `louvain` (left) and `walktrap` (right), for all test datasets.]

Fig. 6.4: Asymmetric evaluation: SR and NMI when the agent trained on `greedy` is tested on `louvain` (left) and `walktrap` (right), for all test datasets.

7. CONCLUSIONS AND FUTURE WORK

This chapter closes the thesis. Section 7.1 summarizes the contributions and the main experimental findings. Section 7.2 discusses the limitations of the current work, and Section 7.3 outlines the most promising directions for future research.

7.1 Summary

In this thesis we studied the Community Membership Hiding problem and identified a critical limitation in the state-of-the-art DRL approach: the original agent is unaware of the target node's identity, learning a generic community-shuffling policy rather than a targeted hiding strategy. We proposed a novel architecture that addresses this limitation through four contributions: (i) target-aware node features with a binary indicator; (ii) a GATv2 encoder with dynamic, query-conditioned attention; (iii) a Dual-Agent Architecture that separates edge addition and deletion into specialized networks; and (iv) a Policy-Guided Subgraphing heuristic for scalability on large graphs.

Experimental results show that each component contributes positively to overall performance, and the full architecture consistently outperforms existing methods across datasets, detection algorithms, and parameter configurations.

7.2 Limitations

Several limitations remain in the current work:

- The method assumes **complete graph knowledge**. In practice, a user may only have access to a local view of the network (e.g., their ego-network).
- Only **non-overlapping communities** are considered. Many real-world networks exhibit overlapping community structure.
- The **black-box assumption** means that knowing the algorithm would enable more targeted strategies; the white-box setting remains unexplored.
- The rewiring budget is defined as a multiple of the mean degree, which may be an unrealistic model of the actual modifications available to a user.

7.3 Future Work

Several directions for future research are natural extensions of this work:

Overlapping communities. Extending the problem formulation and the architecture to handle overlapping communities would significantly broaden the applicability of the method.

Partial graph knowledge. A more realistic setting assumes that the hider has access only to a local ego-network or a sampled subgraph. Designing agents that are robust to partial observations is an important open problem, possibly addressed through belief-state methods or model-based RL.

White-box hiding. If the community detection algorithm is known, one could exploit its structure (e.g., the modularity landscape for Louvain, or the random-walk transition matrix for Walktrap) to design more efficient hiding strategies, or even compute exact gradients through the algorithm.

Multiple target nodes. An extension to the simultaneous hiding of multiple nodes, possibly coordinated via Multi-Agent Reinforcement Learning (MARL), is a natural next step. This setting raises new questions about cooperative strategies and shared budgets.

Synthetic and trivial datasets. Testing the architecture on synthetic graphs with known community structure (e.g., stochastic block models, planted partition models) would provide controlled ablation studies and theoretical insights into when and why the method succeeds or fails.

Certified hiding. An interesting open question is whether one can derive provable guarantees on hiding success, analogous to certified robustness in adversarial machine learning.

BIBLIOGRAPHY

[1] Albert-László Barabási and Réka Albert. "Emergence of Scaling in Random Networks". In: *Science* 286.5439 (1999), pp. 509–512.

[2] Andrea Bernini, Fabrizio Silvestri, and Gabriele Tolomei. "Evading Community Detection via Counterfactual Neighborhood Search". In: *Proceedings of the ACM Web Conference 2024 (WWW '24)*. 2024.

[3] Vincent D. Blondel et al. "Fast unfolding of communities in large networks". In: *Journal of Statistical Mechanics: Theory and Experiment* 2008.10 (2008), P10008.

[4] Shaked Brody, Uri Alon, and Eran Yahav. "How Attentive are Graph Attention Networks?" In: *International Conference on Learning Representations (ICLR)* (2022).

[5] Jinyin Chen et al. "GA Based Q-Attack on Community Detection". In: *IEEE Transactions on Computational Social Systems*. 2019.

[6] Aaron Clauset, M. E. J. Newman, and Cristopher Moore. "Finding community structure in very large networks". In: *Physical Review E* 70.6 (2004), p. 066111.

[7] Hanjun Dai et al. "Adversarial Attack on Graph Structured Data". In: *Proceedings of the 35th International Conference on Machine Learning (ICML)*. 2018, pp. 1115–1124.

[8] Michael Fire, Roy Goldschmidt, and Yuval Elovici. "Online Social Networks: Threats and Solutions". In: *IEEE Communications Surveys & Tutorials* 16.4 (2014), pp. 2019–2036.

[9] Santo Fortunato. "Community detection in graphs". In: *Physics Reports* 486.3–5 (2010), pp. 75–174.

[10] Linton C. Freeman. "A Set of Measures of Centrality Based on Betweenness". In: *Sociometry* 40.1 (1977), pp. 35–41.

[11] Justin Gilmer et al. "Neural Message Passing for Quantum Chemistry". In: *Proceedings of the 34th International Conference on Machine Learning (ICML)*. 2017, pp. 1263–1272.

[12] Aditya Grover and Jure Leskovec. "node2vec: Scalable Feature Learning for Networks". In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016, pp. 855–864.

[13] Paul Jaccard. "Distribution de la flore alpine dans le bassin des Dranses et dans quelques régions voisines". In: *Bulletin de la Société Vaudoise des Sciences Naturelles* 37 (1901), pp. 241–272.

[14] Thomas N. Kipf and Max Welling. "Semi-Supervised Classification with Graph Convolutional Networks". In: *International Conference on Learning Representations (ICLR)*. 2017.

[15] Nina Mazyavkina et al. "Reinforcement Learning for Combinatorial Optimization: A Survey". In: *Computers & Operations Research* 134 (2021), p. 105400.

[16] Tomas Mikolov et al. “Distributed Representations of Words and Phrases and their Compositionality”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2013, pp. 3111–3119.

[17] Volodymyr Mnih et al. “Asynchronous Methods for Deep Reinforcement Learning”. In: *Proceedings of the 33rd International Conference on Machine Learning (ICML)* (2016), pp. 1928–1937.

[18] Arvind Narayanan and Vitaly Shmatikov. “De-anonymizing Social Networks”. In: *IEEE Symposium on Security and Privacy* (2009), pp. 173–187.

[19] M. E. J. Newman. “Modularity and community structure in networks”. In: *Proceedings of the National Academy of Sciences* 103.23 (2006), pp. 8577–8582.

[20] M. E. J. Newman. *Networks: An Introduction*. Oxford University Press, 2010.

[21] Pascal Pons and Matthieu Latapy. “Computing communities in large networks using random walks”. In: *Journal of Graph Algorithms and Applications* 10.2 (2006), pp. 191–218.

[22] Franco Scarselli et al. “The Graph Neural Network Model”. In: *IEEE Transactions on Neural Networks* 20.1 (2009), pp. 61–80.

[23] Fabrizio Silvestri, Gabriele Tolomei, and Andrea Bernini. *The Right to Hide: Masking Community Membership*. 2025. arXiv: `2501.05120 [cs.SI]`.

[24] Thorvald Sørensen. “A method of establishing groups of equal amplitude in plant sociology”. In: *Biologiske Skrifter* 5 (1948), pp. 1–34.

[25] Alexander Strehl and Joydeep Ghosh. “Cluster Ensembles — A Knowledge Reuse Framework for Combining Multiple Partitions”. In: *Journal of Machine Learning Research* 3 (2003), pp. 583–617.

[26] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd. The MIT Press, 2018.

[27] Chao Tang et al. “Reinforcement Learning on Graphs: A Survey”. In: *arXiv preprint arXiv:2204.06127* (2021).

[28] Petar Veličković et al. “Graph Attention Networks”. In: *International Conference on Learning Representations (ICLR)*. 2018.

[29] Duncan J. Watts and Steven H. Strogatz. “Collective dynamics of ‘small-world’ networks”. In: *Nature* 393 (1998), pp. 440–442.

[30] Ronald J. Williams. “Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning”. In: *Machine Learning* 8 (1992), pp. 229–256.

[31] Keyulu Xu et al. “How Powerful are Graph Neural Networks?” In: *International Conference on Learning Representations (ICLR)*. 2019.

[32] Daniel Zügner, Amir Akbarnejad, and Stephan Günnemann. “Adversarial Attacks on Neural Networks for Graph Data”. In: *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2018, pp. 2847–2856.

APPENDIX

. HYPERPARAMETER DETAILS

This appendix lists the full set of hyperparameters used in the experiments. All values were selected by grid search on the `words` validation split, unless noted otherwise.

Tab. .1: Hyperparameters for the proposed architecture.

Hyperparameter	Value
Learning rate (Adam)	7×10^{-4}
Discount factor γ	—
Entropy coefficient c_e	—
Value coefficient c_v	—
GATv2 layers	—
GATv2 hidden dim	—
Attention heads	—
Actor MLP hidden dim	—
Critic MLP hidden dim	—
Training episodes	10,000
Subgraph hops K	2
Reward trade-off α	—

. DATASET DESCRIPTIONS

This appendix provides additional information about the five benchmark datasets used in the evaluation.

words. A co-occurrence network of words extracted from a corpus of English text. Nodes represent words and edges connect words that co-occur in the same sentence. This dataset is used exclusively for training.

kar (Karate Club). The Zachary Karate Club network [20], a classic benchmark for community detection. It contains 34 nodes representing members of a university karate club, with edges corresponding to social interactions outside the club.

vote. A political voting network where nodes represent legislators and edges represent aligned voting records.

pow (Power Grid). A high-voltage power grid network, where nodes are generators, transformers, and substations, and edges are power supply lines. This network has a fundamentally different structure from social networks, providing a heterogeneous test of generalization.

fb-75. A subgraph of the Facebook ego network dataset, selected for its larger size relative to the other benchmarks. This dataset is the primary testbed for the scalability evaluation of Policy-Guided Subgraphing.

. SUPPLEMENTARY RESULTS

This appendix contains full result tables for all combinations of τ, β_{mul}, dataset, and detection algorithm, complementing the summary results presented in Chapter 6.
[Tables to be added upon completion of the full experimental run.]

[Figure placeholder: Full SR results across all datasets, τ values, budget multipliers, and detection algorithms (symmetric and asymmetric settings).]

Fig. .1: Full SR results across all datasets, τ values, budget multipliers, and detection algorithms (symmetric and asymmetric settings).

[Figure placeholder: Full NMI results across all datasets, τ values, budget multipliers, and detection algorithms.]

Fig. .2: Full NMI results across all datasets, τ values, budget multipliers, and detection algorithms.